

Bài 5: NGOẠI LỆ VÀ XỬ LÝ NGOẠI LỆ

Mục tiêu

Về kiến thức:

- Giới thiệu nguyên lý và ý nghĩa của việc xử lý ngoại lệ

Về kỹ năng:

Giúp Sinh viên học cách sử dụng các kỹ thuật xử lý ngoại lệ trong Java để giải quyết các bài toán cụ thể, đồng thời hiểu và áp dụng các phương pháp xử lý ngoại lệ hiệu quả.

5.1. Giới thiệu

Mọi đoạn chương trình đều tiềm ẩn khả năng sinh lỗi. Lỗi chủ quan: do lập trình sai, lỗi khách quan: do dữ liệu, do trạng thái của hệ thống.

Ngoại lệ: các trường hợp hoạt động không bình thường của chương trình ví dụ: đọc 1 file không tồn tại, lỗi chia cho 0 với số nguyên, truy cập vượt quá phạm vi xác định của mảng vv... Xử lý ngoại lệ như thế nào? Làm thế nào để có thể tiếp tục thực hiện?

Cách xử lý lỗi truyền thống: Cài đặt mã xử lý tại nơi phát sinh ra lỗi, làm cho chương trình trở nên khó hiểu, không phải lúc nào cũng đầy đủ thông tin để xử lý, không nhất thiết phải xử lý, truyền trạng thái lên mức trên, thông qua tham số, giá trị trả lại hoặc biến tổng thể (flag). Dễ nhầm, khó kiểm soát được hết các trường hợp, lỗi số học, lỗi bộ nhớ,... Lập trình viên thường quên không xử lý lỗi do bản chất con người, thiếu kinh nghiệm, cố tình bỏ qua.

Ví dụ, việc chia cho 0 sẽ tạo một lỗi trong chương trình hoặc khi xét thao tác nhập xuất (I/O) trong một tập tin. Nếu việc chuyển đổi kiểu dữ liệu không thực hiện đúng, một ngoại lệ sẽ xảy ra và chương trình bị hủy mà không đóng lại tập tin. Lúc đó tập tin dễ bị hư hại và các nguồn tài nguyên được cấp phát cho tập tin không được thu hồi lại cho hệ thống.

Ví dụ chương trình chia 2 số. Nếu ta cho mẫu số = 0 thì phát sinh lỗi và đó được coi là 1 ngoại lệ.

```
public class ExceptionExample1 {  
  
    public static void main(String[] args) {
```

```
int zero = 0;

int average = 10 / zero;

System.out.println("Average = " + average);

}

}
```

Khi thực thi chương trên, sẽ nhận được thông báo lỗi như sau:

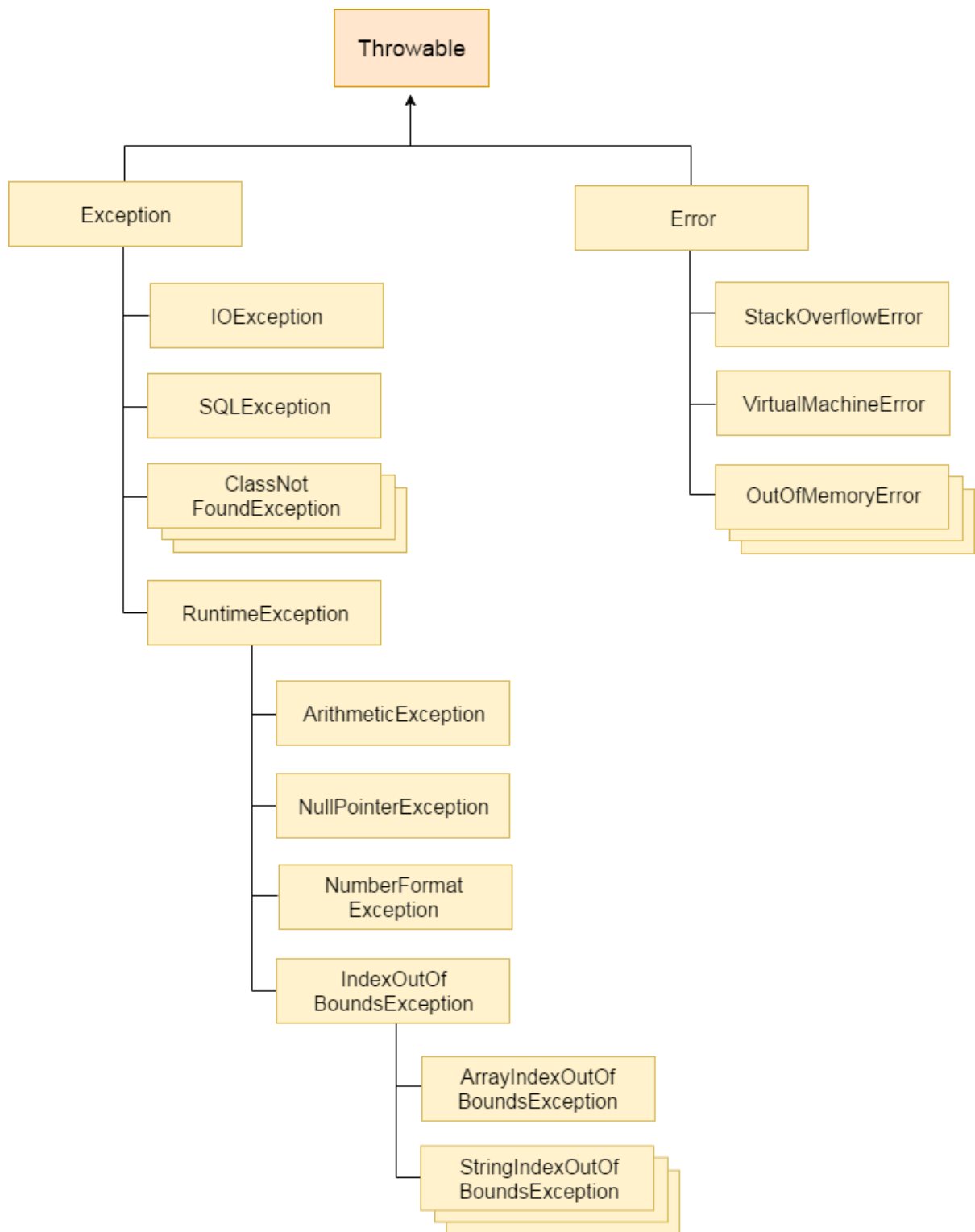
Exception in thread "main" java.lang.ArithmeticException: / by zero

Ngôn ngữ Java cung cơ chế tự động dừng cho bắt và xử lý ngoại lệ. Có hai cách để xử lý ngoại lệ: Bắt, xử lý ngoại lệ và *ném* ngoại lệ (chuyển hướng hay ủy quyền xử lý ngoại lệ đến một vị trí khác của chương trình).

Ưu điểm của bắt ngoại lệ

- Dễ sử dụng
- Dễ dàng chuyển điều khiển đến nơi có khả năng xử lý ngoại lệ
- Có thể ném nhiều loại ngoại lệ
- Tách xử lý ngoại lệ khỏi thuật toán
- Tách mã xử lý
- Sử dụng cú pháp khác
- Không bỏ sót ngoại lệ (ném tự động)
- Làm chương trình dễ đọc hơn, an toàn hơn

5.2. Cấu trúc phân cấp của các lớp xử lý ngoại lệ



Hình 2.5 Ngoại lệ

Ngoại lệ	Lớp cha của thứ tự phân cấp ngoại lệ
RuntimeException	Lớp cơ sở cho nhiều ngoại lệ java.lang
ArithmeticException	Trạng thái lỗi về số, ví dụ như „chia cho 0“

IllegalAccessException	Lớp không thể truy cập
IllegalArgumentException	Phương thức nhận một đối số không hợp lệ
ArrayIndexOutOfBoundsException	Kích thước của mảng lớn hơn 0 hay lớn hơn kích thước thật sự của mảng
NullPointerException	Khi muốn truy cập đối tượng null
SecurityException	Việc thiết lập cơ chế bảo mật không được hoạt động
ClassNotFoundException	Không thể nạp lớp yêu cầu
NumberFormatException	Việc chuyển đổi không thành công từ chuỗi sang số thực
AWTException	Ngoại lệ về AWT
IOException	Lớp cha của các ngoại lệ I/O
FileNotFoundException	Không thể định vị tập tin
EOFException	Kết thúc một tập tin
NoSuchMethodException	Phương thức yêu cầu không tồn tại
InterruptedException	Khi một luồng bị ngắt

Bảng. 2.1 Ngoại lệ

Trong Java có 2 loại exception: checked và unchecked. Tất cả các checked exception được kế thừa từ lớp Exception ngoại trừ lớp RuntimeException. RuntimeException là lớp cơ sở của tất cả các lớp unchecked exception. Đó cũng là dấu hiệu để nhận biết đâu là checked exception và đâu là unchecked exception.

Điểm khác biệt giữa các lớp checked và unchecked exception chính là thời điểm xác định được exception có thể xảy ra.

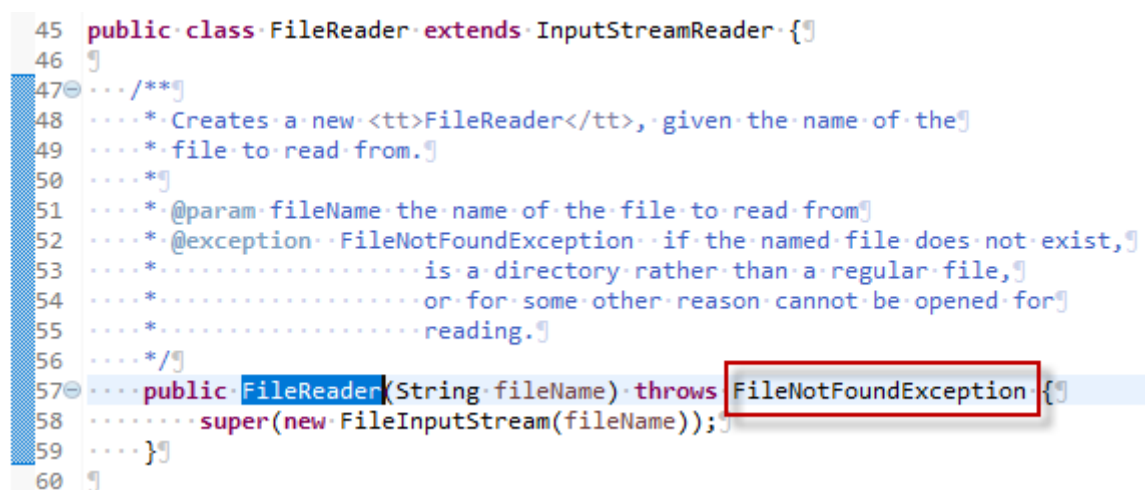
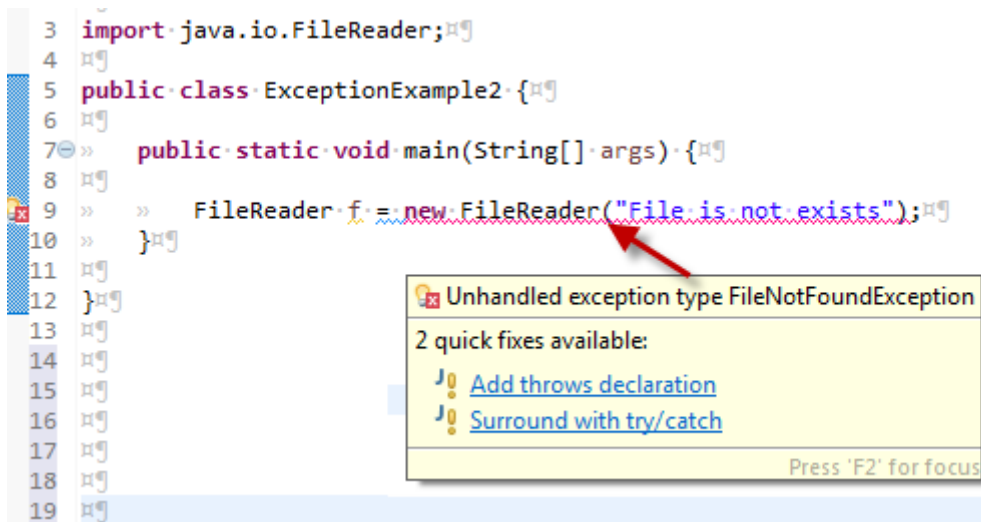
Checked exceptions

Là loại exception xảy ra trong lúc compile time, nó cũng có thể được gọi là **compile time exceptions**. Loại exception này không thể bỏ qua được trong quá trình compile, bắt buộc ta phải handle nó.

Các lớp extends từ lớp Throwable ngoại trừ RuntimeException và Error được gọi là checked exception.

Ví dụ: IOException, FileNotFoundException, NoSuchFieldException,

Ví dụ chương trình sau đọc file sử dụng java.io.FileReader lớp này ném ra ngoại lệ FileNotFoundException. Trình biên dịch thông báo lỗi như sau:



Unchecked exceptions

Là loại exception xảy ra tại thời điểm thực thi chương trình, nó cũng có thể gọi là **runtime exceptions** đó là programming bugs, lỗi logic của chương trình... Loại exception này được bỏ qua trong quá trình compile, không bắt buộc ta phải handle nó. Các lớp extends từ RuntimeException được gọi là unchecked exception.

Ví dụ: NullPointerException, NumberFormatException,

ArrayIndexOutOfBoundsException, DivideByZeroException, ...

Ví dụ một biến có giá trị null, thực hiện bất kỳ hoạt động nào bởi biến đó sẽ xảy ra ngoại lệ NullPointerException.

```
Exception in thread "main" java.lang.NullPointerException
    at com.gpcoder.exception.ExceptionExample3.main(ExceptionExample3.java:10)
```

Error

Error là những vấn đề nghiêm trọng liên quan đến môi trường thực thi của ứng dụng hoặc hệ thống mà lập trình viên không thể kiểm soát. Nó thường làm chết chương trình. Lớp Error định nghĩa các ngoại lệ mà không thể bắt (catch) từ chương trình.

Ví dụ: OutOfMemoryError, VirtualMachineError, and StackOverflowError, ...

Ví dụ chương trình đệ quy vô tận:

```
3 public class ExceptionExample4 {  
4     »  
5     » public static void print() {  
6     »     » print();  
7     »     }  
8     »  
9     » public static void main(String[] args) {  
10    »     » print();  
11    »     }  
12    »  
13    » }
```

```
Exception in thread "main" java.lang.StackOverflowError  
    at com.gpcoder.exception.ExceptionExample4.print(ExceptionExample4.java:6)  
    at com.gpcoder.exception.ExceptionExample4.print(ExceptionExample4.java:6)  
    at com.gpcoder.exception.ExceptionExample4.print(ExceptionExample4.java:6)  
    at com.gpcoder.exception.ExceptionExample4.print(ExceptionExample4.java:6)  
    at com.gpcoder.exception.ExceptionExample4.print(ExceptionExample4.java:6)
```

5.3. Xử lý ngoại lệ (Exception Handling) trong java

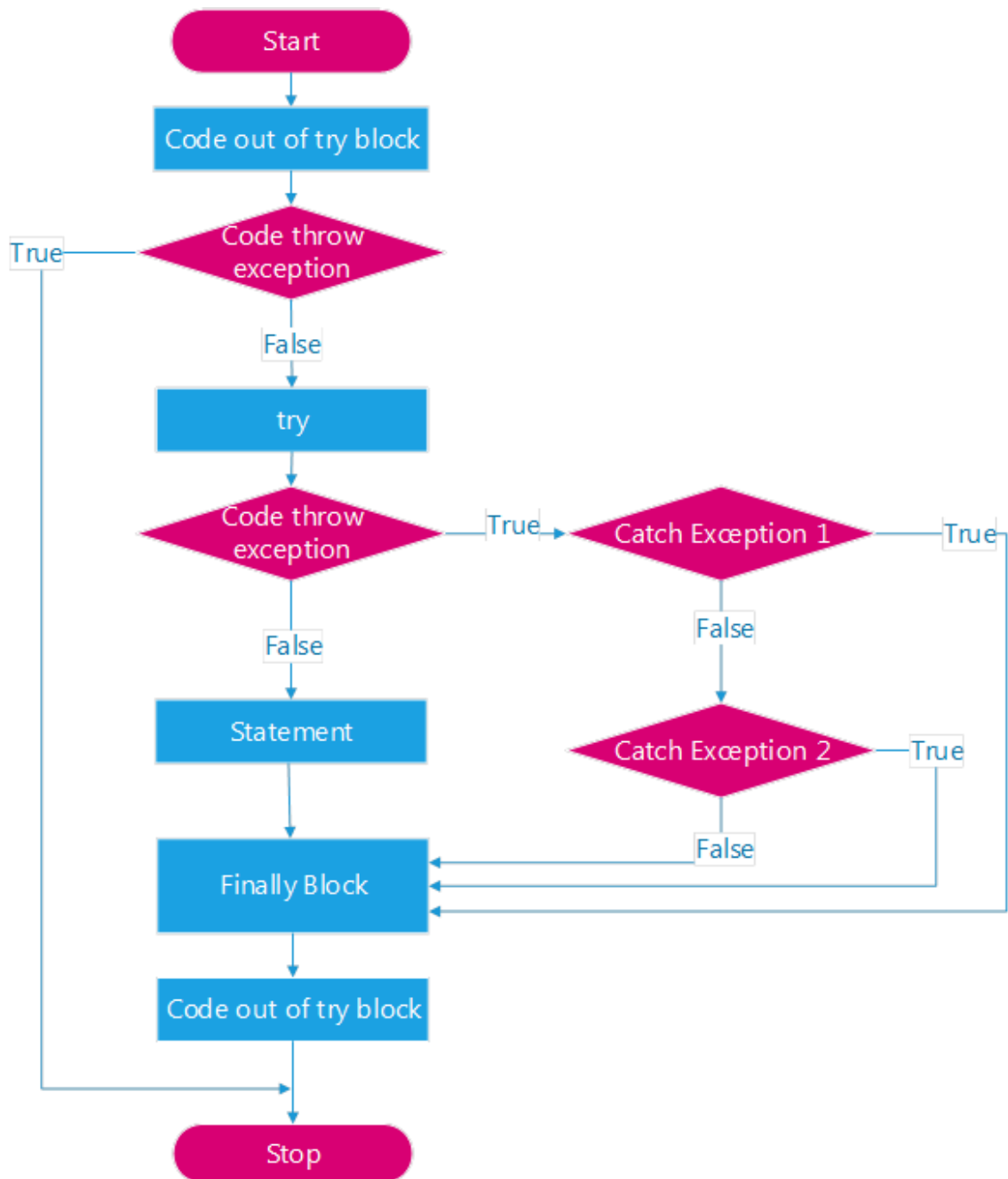
Xử lý ngoại lệ (Exception Handling) trong java là một cơ chế xử lý các lỗi runtime để có thể duy trì luồng bình thường của ứng dụng.

Quá trình xử lý exception được gọi là catch exception, nếu Runtime System không xử lý được ngoại lệ thì chương trình sẽ kết thúc.



Khi một lỗi xảy ra trên một method, method đó sẽ tạo ra một object và đưa nó vào **Runtime System**. Object đó được gọi là Exception Object, nó chứa tất cả các thông tin về lỗi và trạng thái của chương trình khi xảy ra lỗi.

Sau đó, Runtime System sẽ xử lý sẽ tìm cách xử lý ngoại lệ phù hợp được sử dụng tại method ấy. Nếu không có thì JVM tiếp tục tìm xử lý ngoại lệ phù hợp ở các method trên (là method gọi lớp hiện tại). Nếu không có method nào có xử lý ngoại lệ phù hợp thì Thread mà đang thực hiện chuỗi method xảy ra ngoại lệ bị ngắt. Nếu thread ấy là thread main thì chết chương trình.



5.4. Câu lệnh try, catch và finally

```

try { // Khởi try
<Các câu lệnh> }
catch (<Kiểu ngoại lệ 1> <Tham biến 1>) { // Khởi catch
<Các câu lệnh xử lý khi xuất hiện kiểu ngoại lệ 1> }
...
catch (<Kiểu ngoại lệ n> <Tham biến n>) { // Khởi catch
<Các câu lệnh xử lý khi xuất hiện kiểu ngoại lệ n>

```

}

finally { < Các câu lệnh phải thực hiện đến cùng> // Khối finally }

Trong đó:

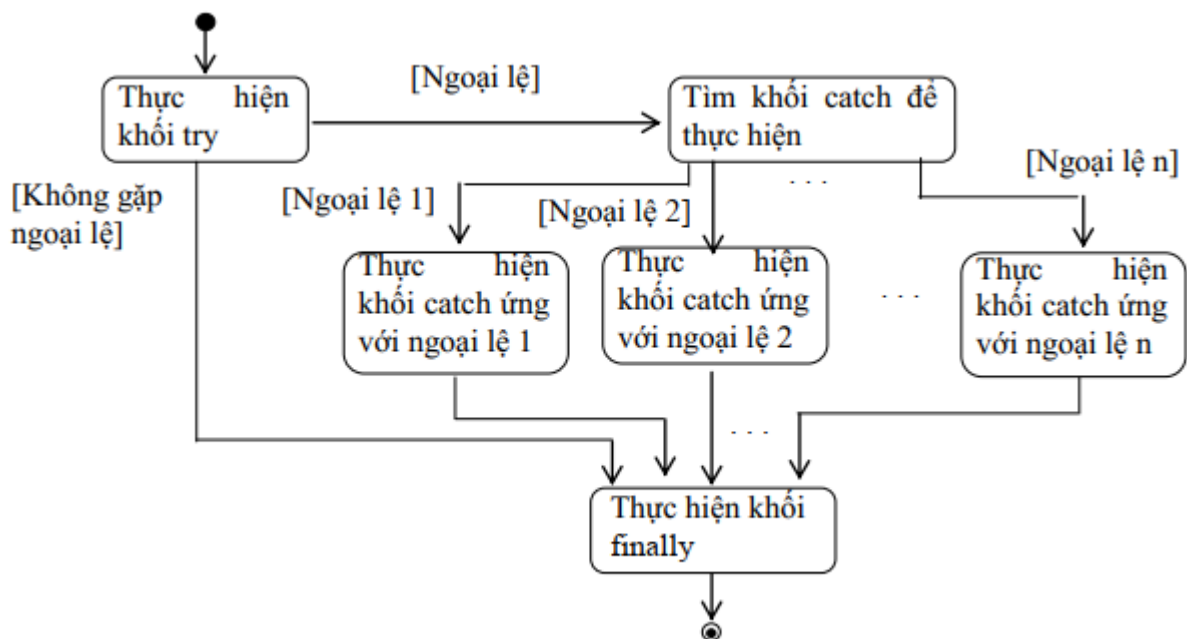
- <Các câu lệnh> là các câu lệnh có khả năng gây ra ngoại lệ

- <Kiểu ngoại lệ> là các kiểu ngoại lệ tương ứng.

<tham biến i> là đối tượng mang thông tin về ngoại lệ

Các ngoại lệ sẽ được cho qua trong quá trình thực hiện khối *try* và sẽ bị tóm lại để xử lý ở các khối *catch* tương ứng nếu có. Khối *finally* phải thực hiện đến cùng, bất luận có gặp phải ngoại lệ hay không.

Hoạt động của các khối trên được minh họa như sau:



Hình 2.6. Try catch

Khối try

Một chương trình cũng có thể chứa các khối *try* lồng nhau. Khi sử dụng các “*try*” lồng nhau, khối *try* bên trong được thi hành đầu tiên. Bất kỳ ngoại lệ nào bị chặn trong khối *try* sẽ bị bắt giữ trong các khối *catch* theo sau. Nếu khối *catch* thích hợp không được tìm thấy thì các khối *catch* của các khối *try* bên ngoài sẽ được xem xét. Nếu không, Java Runtime Environment xử lý các ngoại lệ.

Khối catch

Khối này chỉ được sử dụng để xử lý ngoại lệ nếu có. Khi một khối *catch* được thực hiện thì các khối *catch* còn lại sẽ bị bỏ qua.

Có thể bắt nhiều loại ngoại lệ khác nhau bằng cách sử dụng nhiều khối lệnh *catch* đặt kế tiếp

Khối lệnh *catch* sau không thể bắt ngoại lệ là lớp dẫn xuất của ngoại lệ được bắt trong khối lệnh *catch* trước.

Khối *finally*

Khi một ngoại lệ xuất hiện, phương thức đang được thực thi có thể bị dừng mà không được thi hành toàn vẹn. Nếu điều này xảy ra, thì các đoạn mã sẽ không bao giờ được gọi. Khối *finally* thực hiện tất cả các việc thu dọn khi một ngoại lệ xảy ra. Khối *finally* bảo đảm lúc nào cũng được thực thi, bất chấp có ngoại lệ xảy ra hay không. Khối *finally* là tùy ý, không bắt buộc.

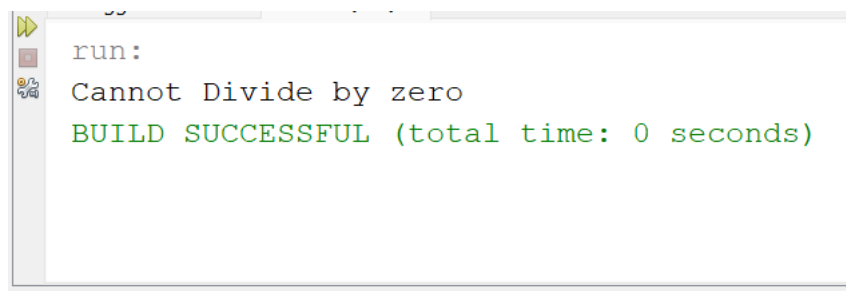
Khối *finally* có thể dùng để:

- Đóng tập tin.
- Đóng lại bộ kết quả (sử dụng trong chương trình cơ sở dữ liệu).
- Đóng lại các kết nối được tạo trong cơ sở dữ liệu.

Ví dụ minh họa cách sử dụng các khối *try* và *catch*.

```
class TryClass{  
    public static void main(String args[])  
    {  
        int demo=0;  
        try { System.out.println(20/demo); }  
        catch(ArithmeticException a) {  
            System.out.println("Cannot Divide by zero"); }  
    }  
}
```

Kết quả:



```
run:  
Cannot Divide by zero  
BUILD SUCCESSFUL (total time: 0 seconds)
```

a được sử dụng như một đối tượng của *ArithmeticException* để in các chi tiết về các toán tử ngoại lệ mà hệ thống cung cấp. Nếu bạn thay thế lệnh *System.out.println* của khối *catch* bằng lệnh *System.out.println(a.getMessage())* thì kết quả của chương trình sẽ như sau:

Các phương thức của lớp Exception

STT	Phương thức và Miêu tả
	public String getMessage()
1	Trả về một message cụ thể về exception đã xảy ra. Message này được khởi tạo bởi phương thức constructor của Throwable
	public Throwable getCause()
2	Trả về nguyên nhân xảy ra exception biểu diễn bởi đối tượng Throwable
	public String toString()
3	Trả về tên của lớp và kết hợp với kết quả từ phương thức getMessage()
	public void printStackTrace()
4	In ra kết quả của phương thức toString cùng với stack trace đến System.err
	public StackTraceElement [] getStackTrace()
5	Trả về một mảng chứa mỗi phần tử trên stack trace. Phần tử tại chỉ mục 0 biểu diễn phần trên cùng của Call Stack, và phần tử cuối cùng trong mảng biểu diễn phương thức tại dưới cùng của Call Stack
	public Throwable fillInStackTrace()
6	Fills the stack trace of this Throwable object with the current stack trace, adding to any previous information in the stack trace.

Ví dụ xử lý ngoại lệ thực hiện phép chia cho số 0

Khi chưa xử lý ngoại lệ

```
public class ExceptionExample1 {  
  
    public static void main(String[] args) {  
        int zero = 0;  
        int average = 10 / zero;  
        System.out.println("Average = " + average);  
    }  
}
```

```
        System.out.println("Finished!");  
    }
```

```
}
```

Thực thi chương trình trên:

Exception in thread "main" java.lang.ArithmeticException: / by zero
at com.gpcoder.exception.ExceptionExample1.main(ExceptionExample1.java:7)

Xử lý ngoại lệ ArithmeticException cho chương trình trên

```
public class ExceptionExample1 {  
    public static void main(String[] args) {  
        try {  
            int zero = 0;  
            int average = 10 / zero;  
            System.out.println("Average = " + average);  
        } catch (ArithmeticException ex) {  
            System.out.println(ex);  
        }  
        System.out.println("Finished!");  
    }  
}
```

Kết quả thực thi chương trình trên:

java.lang.ArithmeticException: / by zero
Finished!

Lưu ý:

- Tại một thời điểm chỉ xảy ra một ngoại lệ và tại một thời điểm **chỉ có một khối catch được thực thi**. Khi exception đã bị bắt ở một catch thì các catch tiếp theo sẽ không được bắt.
- Tất cả các khối catch phải được sắp xếp **từ cụ thể nhất đến chung nhất (từ exception con đến exception cha)**, tức là phải khai báo khối lệnh catch để xử lý lỗi NullPointerException, ArithmeticException, ... trước khi khai báo catch để xử lý lỗi Exception.
- Khối lệnh finally luôn được thực thi dù chương trình có xảy ra ngoại lệ hay không (ngay cả sử dụng lệnh **return**).
- Đối với mỗi khối try, có thể không có hoặc nhiều khối catch, nhưng chỉ có một khối finally.
- Khối finally sẽ không được thực thi nếu chương trình bị thoát bằng cách gọi **System.exit()** hoặc xảy ra một lỗi (Error) không thể tránh khiến chương trình bị chết.

5.5. Ném ngoại lệ bằng lệnh ‘throw’

Để tạm thời bỏ qua ngoại lệ chúng ta có thể sử dụng câu lệnh *throw*. Đoạn lệnh sau chỉ ra cách sử dụng của lệnh *throw*:

```
try{ if (flag<0)
    {throw new MyException();} // user-defined
}
```

Ví dụ:

```
public class VD_NgoaiLe2 {
    public static void main(String[] args) {
        try {
            checkAge(15);
        } catch (IllegalArgumentException e) {
            System.out.println("Ngoại lệ xảy ra: " + e.getMessage());
        }
    }

    public static void checkAge(int age) {
        if (age < 18) {
            throw new IllegalArgumentException("Tuổi phải từ 18 trở lên để
            đăng ký.");
        } else {
            System.out.println("Đăng ký thành công!");
        }
    }
}
```

Chạy chương trình sẽ in ra:

Ngoại lệ xảy ra: Tuổi phải từ 18 trở lên để đăng ký.

Giả sử phương thức *x()* gọi phương thức *y()*. Phương thức *y()* chặn một ngoại lệ không được xử lý. Phương thức gọi *x()* nên khai báo việc chặn cùng một ngoại lệ với phương thức được gọi *y()*. Ta nên khai báo khối *try catch* trong phương thức *x()* để đảm bảo rằng ngoại lệ không được truyền cho các phương thức mà gọi phương thức này.

Phương thức được định nghĩa lại tại lớp dẫn xuất có thể không ném ngoại lệ. Nếu ném ngoại lệ, chỉ có thể ném ngoại lệ giống tại phương thức của lớp cơ sở hoặc ngoại lệ là lớp dẫn xuất của ngoại lệ được ném tại phương thức của lớp cơ sở.

Ví dụ:

```

class A {public void methodA() throws RuntimeException { }
}
class B extends A {
public void methodA() throws ArithmeticException { }
}
class C extends A {
public void methodA() throws Exception { }
}
class D extends A {
public void methodA() { }
}
A a = new B();
try { a.methodA(); }
catch (RuntimeException e) { ...}

```

Ném lại ngoại lệ: Sau khi bắt ngoại lệ, nếu thấy cần thiết chúng ta có thể ném lại chính ngoại lệ vừa bắt được để cho chương trình mức trên tiếp tục xử lý.

```

try {...}
catch (Exception e) {
System.out.println(e.getMessage());
throw e;}

```

5.6. Mệnh đề throws

Khi thiết kế các hàm thành phần, chúng ta có thể sử dụng mệnh đề throws để tạm thời cho qua ngoại lệ mà thực hiện một số công việc cần thiết khác. Chúng ta ném ngoại lệ khỏi phương thức khi:

- Không nhất thiết phải xử lý ngoại lệ trong phương thức
- Không đủ thông tin để xử lý
- Không đủ thẩm quyền

Định nghĩa hàm với mệnh đề *throws* có dạng:

```

<Thuộc tính của hàm> <tên hàm>(<Danh sách các tham biến>)
throws <Danh sách các kiểu ngoại lệ> { /* ... */}

```

Trong đó <Danh sách các kiểu ngoại lệ> là các lớp xử lý ngoại lệ được kế thừa từ lớp Exception và được phân tách bởi dấu ‘,’.

Ném ngoại lệ từ main()

– Nếu không có phương thức nào bắt ngoại lệ, ngoại lệ sẽ được truyền lên phương thức main() và được cần được xử lý tại đây.

– Nếu vẫn không muốn xử lý ngoại lệ, chúng ta có thể để ngoại lệ truyền lên mức điều khiển của máy ảo bằng cách khai báo main() ném ngoại lệ

– chương trình sẽ bị dừng và hệ thống sẽ in thông tin về ngoại lệ trên Console (printStackTrace())

Ví dụ:

```
public class VD_NgoaiLe2 {  
    public static void main(String[] args) {  
        try {  
            divide(10, 0);  
        } catch (ArithmeticException e) {  
            System.out.println("Không thể chia cho số 0!");  
        }  
    }  
  
    public static void divide(int a, int b) throws ArithmeticException {  
        int result = a / b;  
        System.out.println("Kết quả là: " + result);  
    }  
}
```

So sánh sự khác nhau giữa từ khóa throw và throws trong java được mô tả trong bảng dưới đây:

	throw	throws
1)	Từ khóa throw trong java được sử dụng để ném ra một ngoại lệ rõ ràng.	Từ khóa throws trong java được sử dụng để khai báo một ngoại lệ.
2)	Ngoại lệ checked không được truyền ra nếu chỉ sử dụng từ khóa throw.	Ngoại lệ checked được truyền ra ngay cả khi chỉ sử dụng từ khóa throws.
3)	Sau throw là một instance.	Sau throws là một hoặc nhiều class.
4)	throw được sử dụng trong phương thức.	throws được khai báo ngay sau dấu đóng ngoặc đơn của phương thức.

5)	Bạn không thể throw nhiều exceptions.	Bạn có thể khai báo nhiều exceptions, Ví dụ: public void method()throws IOException,SQLException.
----	---------------------------------------	---

5.7. Tự định nghĩa ngoại lệ

Chúng ta có thể tạo lớp ngoại lệ để phục vụ các mục đích riêng

- Lớp ngoại lệ mới phải kế thừa từ lớp Exception hoặc lớp dẫn xuất của lớp này.

- Có thể cung cấp hai constructor

- constructor nhận một tham số String và truyền tham số này cho phương thức khởi tạo của lớp cơ sở.

Ví dụ:

```
class SimpleException extends Exception { }
class MyException extends Exception {
    public MyException() {}
    public MyException(String msg) {
        super(msg);}
}
```

Mình họa ngoại lệ được định nghĩa bởi người dùng *ArraySizeException*:

```
class ArraySizeException extends NegativeArraySizeException
{
    ArraySizeException() // constructor
    {
        super("You have passed an illegal array size");
    }
}
class ThrowDemo
{
    int size, array[];
    ThrowDemo(int s)
    {
        size=s;
        try {
            checkSize();
        }
        catch(ArraySizeException e){
            System.out.println(e);
        }
    }
    void checkSize() throws ArraySizeException
```

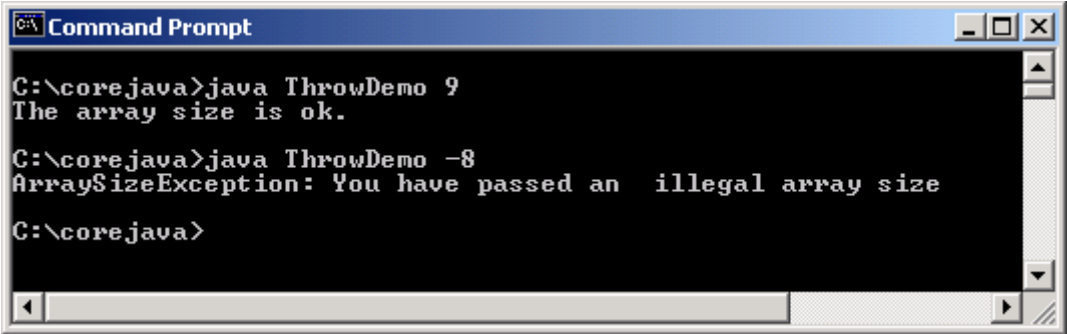
```

{
    if (size < 0)
        throw new ArraySizeException();
    else
        System.out.println("The array size is ok.");
    array = new int[3];
    for (int i=0; i<3; i++)
        array[i] = i+1;
}
public static void main(String arg[])
{
    new ThrowDemo(Integer.parseInt(arg[0]));
}
}

```

Lớp được định nghĩa bởi người dùng *ArraySizeException* là lớp con của lớp *NegativeArraySizeException*. Khi một đối tượng được tạo từ lớp này, thông báo về ngoại lệ được in ra. Phương thức *checkSize()* được gọi để chặn ngoại lệ *ArraySizeException* mà được chỉ ra bởi mệnh đề *throws*. Kích thước của mảng được kiểm tra trong cấu trúc *if*. Nếu kích thước là số âm thì đối tượng của lớp *ArraySizeException* được tạo. Phương thức *call()* được bao quanh trong khối *trycatch*, là nơi mà giá trị của đối tượng được in ra. Phương thức *call()* cần được bao trong khối *try*, để cho khối *catch* tương ứng có thể in ra giá trị.

Kết xuất của chương trình:



```

C:\core.java>java ThrowDemo 9
The array size is ok.

C:\core.java>java ThrowDemo -8
ArraySizeException: You have passed an illegal array size

C:\core.java>

```

5.8. Lăn vết ngoại lệ StackTrace

Có thể sử dụng phương thức *printStackTrace()* để lăn vết vị trí phát sinh ngoại lệ debug chương trình

```

public class Test4 {
    void methodA() throws Exception {
        methodB();
        throw new Exception();
    }
}

```

```

    void methodB() throws Exception {
        methodC();
        throw new Exception();}
    void methodC() throws Exception {
        throw new Exception();
    }
    public static void main(String[] args) {
        Test4 t = new Test4();
        try {
            t.methodA();
        }
        catch(Exception e) {
            e.printStackTrace();}
    }
}

```

5.9. Bài tập mẫu

Bài 1 : Bắt và xử lý ngoại lệ chia cho 0 với số nguyên

```

class Main {
    public static void main(String[] args) {
        try {
            int divideByZero = 5 / 0;
            System.out.println("Rest of code in try block");
        }
        catch (ArithmeticException e) {
            System.out.println("ArithmeticException => " + e.getMessage());
        }
    }
}

```

Bài 2 : Bắt và xử lý ngoại lệ định dạng kiểu số trong java

```

class Main {
    public static void main(String args[]) {
        Scanner input = new Scanner(System.in);
        Integer intNumber = null;
        try {
            System.out.println("Hãy nhập vào một số nguyên: ");
            String strNumber = input.nextLine();
            intNumber = new Integer(strNumber);
        } catch (NumberFormatException e) {
            intNumber=0;
            System.out.println("Lỗi định dạng của số nguyên : "
            +e.getMessage());
        }
        System.out.println("Chuyển thành Hexa: " +
        Integer.toHexString(intNumber));
    }
}

```

}

5.10. Bài tập thực hành

Bài 1: Sử dụng cấu trúc try, và 2 khối catch xây dựng chương trình nhập vào 2 số nguyên a, b và in ra kết quả a/b. Yêu cầu bắt các ngoại lệ `NumberFormatException` khi nhập a và b, `ArithmeticException` khi thực hiện a/b.

Bài 2: Dùng mệnh đề throw để tạo ra ngoại lệ khi nhập tuổi cho nhân viên (không âm và không lớn hơn 62 tuổi)

Bài 3: Xây dựng hàm `nhapTuoi()` và dùng mệnh đề `Throws` để chuyển ngoại lệ ra ngoài hàm và thực hiện bắt và xử lý các ngoại lệ khi gọi hàm `nhapTuoi()`.